

Acceptance Test Procedure

Automated host-side tests and bench acceptance tests

Document number	iW-ASCEJ-01-ATP-R1.0-REL1.0
Revision	R1.0
Package release	REL1.0 (software 1.2.5)
Date	05-Oct-2026
Record of results	iW-ASCEJ-01-ATR-R1.0-REL1.0.xlsx

Prepared by	
Reviewed by	
Approved by	

Contents

Introduction

- Purpose

- Test classes

- Pass criteria

Test environment

Automated tests (TS)

Bench tests (BT)

- BT-01 Package integrity

- BT-02 Firmware boot and ABI

- BT-03 Driver load and device nodes

- BT-04 GUI launch and PCIe pre-flight

- BT-05 RFDC attach, ping and register sync

- BT-06 Boot defaults readback

- BT-07 DDS tone reception

- BT-08 ADC dynamic performance

- BT-09 DAC input source switch

- BT-10 Host transmit

- BT-11 Verify loopback

- BT-12 Transmit guard

- BT-13 Capture throughput per channel count

- BT-14 Start / Stop responsiveness

- BT-15 Helper recovery

- BT-16 Command-line client

- BT-17 Reset all to defaults

- BT-18 Recording and offline replay

- BT-19 Serial menu

- BT-20 Power-cycle persistence

Introduction

Purpose

This procedure defines the acceptance tests for package iW-ASCEJ-01-DF-R1.0-REL1.0 (software 1.2.5) of the iWave SDR platform on the ZU47DR RFSoc. Results are recorded in `iW-ASCEJ-01-ATR-R1.0-REL1.0.xlsx`, in this folder.

Test classes

Class	IDs	Needs hardware	Where it is recorded
Automated host-side	TS-01 ... TS-10	No	ATR sheet <i>Automated Results</i> (filled from <code>01_Logs</code>)
Bench	BT-01 ... BT-20	Yes	ATR sheet <i>Bench Record</i> (filled in by the tester)

Pass criteria

A test passes when every expected result is met. The package is accepted when every automated test passes and every bench test is PASS or N/A (with a justification). Record a failed test with its `sdr_backend.log`, `dmesg` output and the UART log.

Test environment

Item	Reference used for the automated results
Host OS	Ubuntu 24.04.4 LTS, Linux 6.18 x86-64
Compiler	gcc 13.3.0, CMake 3.28.3
Qt	5.15.13
glibc	2.39

Bench equipment: an iWave ZU47DR board with the PL design `H2C_pcie_control`, a PCIe host, a UART console, an SMA loopback cable with attenuation between a DAC output and an ADC input, and optionally a spectrum analyser.

Automated tests (TS)

Run from the package root:

```
iW-ASCEJ-01-SF-R1.0-REL1.0/iW-ASCEJ-01-TS-R1.0-REL1.0/01_Host_Test_Suite/run_all.sh
```

Expected: ALL TESTS: PASS. Each suite builds the code under test straight from SC and FF, so the delivered sources are what is tested.

ID	Test	Suite / ctest name	Log
TS-01	Firmware DAC input routing	<code>firmware/run_tests.sh</code>	<code>01_Logs/TS-01_firmware_dac_routing.log</code>
TS-02	PCIe end-to-end, firmware 1.2.1	<code>pcie_emu / pcie_e2e_fw_1_2_1</code>	<code>01_Logs/TS-02_pcie_emu_pcie_e2e_fw_1_2_1.log</code>
TS-03	PCIe end-to-end, firmware 1.2.0 (not reflashed)	<code>pcie_emu / pcie_e2e_fw_1_2_0</code>	<code>01_Logs/TS-03_pcie_emu_pcie_e2e_fw_1_2_0.log</code>
TS-04	Host transmit path	<code>gui / tx_path</code>	<code>01_Logs/TS-04_gui_tx_path.log</code>
TS-05	Simulated RF loopback	<code>gui / sim_loopback</code>	<code>01_Logs/TS-05_gui_sim_loopback.log</code>
TS-06	H2C / RFDC control window	<code>gui / control_window</code>	<code>01_Logs/TS-06_gui_control_window.log</code>
TS-07	Whole application, simulation	<code>gui / app_loopback</code>	<code>01_Logs/TS-07_gui_app_loopback.log</code>
TS-08	Real iwfg_h2c with emulated card	<code>h2c_emu / h2c_real_helper</code>	<code>01_Logs/TS-08_h2c_emu_h2c_real_helper.log</code>
TS-09	Send-and-receive chain, fault injection	<code>h2c_emu / loopback_chain</code>	<code>01_Logs/TS-09_h2c_emu_loopback_chain.log</code>
TS-10	Whole application, live session, emulated card	<code>h2c_emu / app_live_session</code>	<code>01_Logs/TS-10_h2c_emu_app_live_session.log</code>

Bench tests (BT)

BT-01 Package integrity

Objective: Confirm the delivered package is complete and unmodified.

Set-up: Host with the package extracted.

Procedure:

1. Run `TS/02_Scripts/verify.sh` from the package root.

Expected result: RESULT: PASS: manifest matches, 37 firmware files, 13 contract checks PASS.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-02 Firmware boot and ABI

Objective: Confirm that the complete firmware set is running.

Set-up: Board booted from the BOOT.BIN built from FF; UART0 console at 115200 8N1.

Procedure:

1. Power-cycle the board.
2. Capture the console log.

Expected result: [init] DAC input = DDS compiler and [pcie] pcie_cfg ABI 4 ... caps 0x7F are printed. There are no init errors.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-03 Driver load and device nodes

Objective: Confirm that the iwfg QDMA driver binds the card.

Set-up: Host with kernel headers; card enumerated (`lspci`).

Procedure:

1. `make` in `SC/01_sdr_workstation_GUI/drivers`.
2. `sudo insmod iwfg.ko`.
3. `ls -l /dev/iwfg*; dmesg | grep -i iwfg`.

Expected result: `/dev/iwfg0` and `/dev/iwfg1` exist. There are no driver errors.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-04 GUI launch and PCIe pre-flight

Objective: Confirm the GUI detects the link.

Set-up: GUI built from SC (or BN binary).

Procedure:

1. Start `sdr_workstation`; select Live — PCIe DMA, `/dev/iwfg0`, cs16, 4 channels, 200 MSPS.

Expected result: The link-discovery screen shows the driver, the nodes, and the PCIe link speed and width, then continues on its own. Record the link speed and width.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-05 RFDC attach, ping and register sync

Objective: Confirm the PCIe control mailbox works end to end.

Set-up: GUI running; H2C CONTROL ► RFDC tab.

Procedure:

1. Press Attach.
2. Press Ping.
3. Observe the register readout.

Expected result: Ping is acknowledged. Every register is read back, and the readout reports ABI 4.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-06 Boot defaults readback

Objective: Confirm the board boots at the documented operating point.

Set-up: Freshly powered board; RFDC tab attached.

Procedure:

1. Read the ADC/DAC rates, decimation, interpolation, NCO, DDS, DSA, VOP, DAC source, PL GPIO mode and AXIS routing.

Expected result: 4800 / 9600 MSPS, ×24 / ×24, NCO 3100 MHz, DDS 10 MHz, DSA 0 dB, VOP 32 mA, DAC source = DDS, GPIO

mode 3, routing 0x1237.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-07 DDS tone reception

Objective: Confirm the receive chain with the on-chip source.

Set-up: DAC output looped back to an ADC input through attenuation; DAC source = DDS.

Procedure:

1. Start capture.
2. Read the peak frequency and level.

Expected result: A single tone at the expected baseband frequency (DDS 10 MHz through the 3100 MHz NCOs; see TS-05). Record the level (dBFS).

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-08 ADC dynamic performance

Objective: Record the RF metrics at the boot configuration.

Set-up: As BT-07, with the Blackman-Harris window and FFT 2048 or larger.

Procedure:

1. Read SINAD, SNR, SFDR, THD and ENOB from the Measurements panel.

Expected result: Record all values. The acceptance limit is to be agreed with the customer. Reference: about 11 ENOB bits observed on the bench at the correct configuration.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-09 DAC input source switch

Objective: Confirm DDS ↔ host selection and readback.

Set-up: RFDC tab attached.

Procedure:

1. Set the DAC input to Host, then read back.
2. Set the DAC input to DDS, then read back.
3. Repeat with `rfdc_ctl dac-source host|dds`.

Expected result: The readback matches each selection; the chain view says *confirmed by the device*.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-10 Host transmit

Objective: Confirm that generator samples reach the DAC continuously.

Set-up: DAC source = Host; generator: sine, 10 MHz, amplitude 0.9, Loop-coherent on.

Procedure:

1. Start the generator.
2. Read the H2C to card row.
3. Read the received tone.

Expected result: H2C rate ≈ 762.9 MiB/s. The received tone sits at the coherent frequency (9 999 847 Hz at 1 MiB). There are no broadband disturbances.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-11 Verify loopback

Objective: Confirm the received tone follows the generator.

Set-up: As BT-10, with the looped-back ADC routed to AXIS output 0.

Procedure:

1. Press Verify loopback.

Expected result: ✓ Loopback verified: the received tone moves by the same amount as the generator.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-12 Transmit guard

Objective: Confirm the capture is reduced while transmitting and restored afterwards.

Set-up: Capture 4 channels at 200 MSPS; RFDC tab attached.

Procedure:

1. Start the generator.
2. Read the PL GPIO mode and the capture rate.
3. Stop the generator.

Expected result: While transmitting: GPIO mode 2, capture $\approx$ 1526 MiB/s. After the stop: mode 3 is restored.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-13 Capture throughput per channel count

Objective: Measure the delivered capture rate.

Set-up: Capture-only session.

Procedure:

1. Run 1, 2 and 4 channels (GPIO mode 1/2/3) for 60 s each.
2. Record the MB/s value and `dmesg` overflow messages.

Expected result: 1 ch $\approx$ 763 MiB/s and 2 ch $\approx$ 1526 MiB/s without overflow. Record the 4-channel result (above the host drain on the reference host).

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-14 Start / Stop responsiveness

Objective: Confirm the GUI never blocks.

Set-up: Live session with transmit and capture running.

Procedure:

1. Press Stop, Start and close the window, five times each.

Expected result: Each action returns at once (< 1 s). `sdr_backend.log` shows H2C closed before C2H.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-15 Helper recovery

Objective: Confirm automatic recovery from a helper failure.

Set-up: Live session with transmit and capture running.

Procedure:

1. `pkill c2h_stream`.
2. Observe the GUI and `sdr_backend.log`.

Expected result: The capture helper is restarted and the display re-attaches, with no dialog. Transmit is not interrupted.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-16 Command-line client

Objective: Confirm `rfdc_ctl` controls the board.

Set-up: `rfdc_ctl` from BN or SC.

Procedure:

1. `rfdc_ctl ping`
2. `rfdc_ctl nco-freq both all both 3100`
3. `rfdc_ctl dump`

Expected result: Each command is acknowledged with `0x2A`, error 0. `dump` decodes the last command.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-17 Reset all to defaults

Objective: Confirm recovery to the baseline.

Set-up: Change the decimation, NCO and DSA first.

Procedure:

1. RFDC tab ► Reset all to defaults.
2. Read back as in BT-06.

Expected result: All values return to the BT-06 defaults.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-18 Recording and offline replay

Objective: Confirm that recordings are byte-exact and can be replayed.

Set-up: Live capture, 2 channels.

Procedure:

1. Record for 10 s.
2. Open the file in offline mode.

Expected result: The file size $\approx$ rate $\times$ time. The replay shows the same spectrum, and the `.meta` sidecar restores the format and channel settings.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-19 Serial menu

Objective: Confirm the console functions.

Set-up: UART console.

Procedure:

1. Menu 5 (rates), menu 11 (clocks), menu 17 (PCIe diagnostics).

Expected result: The rates match BT-06. The stream clocks are reported, and the PCIe diagnostics complete.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____

BT-20 Power-cycle persistence

Objective: Confirm the boot defaults return after a power cycle.

Set-up: Change the settings, then power-cycle.

Procedure:

1. Read back as in BT-06.

Expected result: The BT-06 defaults are restored.

Result: ☐ PASS ☐ FAIL ☐ N/A Measured: _____ Tester / date: _____